

UNIVERSITY OF CLAUDE · PROGRAM HANDBOOK · 2026

Master of Science in Computer Science

Graphics & Game Engines track · 36 credit-equivalents · self-paced, entirely free

READ THIS FIRST

The University of Claude is not an accredited institution. This program awards no degree, transcript, or credential, and nothing here is recognised by a registrar anywhere.

What it is: a complete, rigorously sequenced graduate curriculum with original lecture materials, mapped onto free university courseware — so that the *learning* is equivalent even though the paperwork is not. The portfolio you build is the credential.

All material written for this program is original and free to copy, adapt, and redistribute. No institution's copyrighted lecture slides, notes, or problem sets are reproduced; external courses are referenced by link only, and remain under their own licences.

1 What this program is for

A graduate curriculum is not a list of topics. It is a **dependency graph with a deadline**: a sequence in which each course makes the next one tractable, compressed enough that you are still holding the earlier material when the later material needs it. Most self-study fails not from bad resources — the free resources are extraordinary — but from an absent sequence, no forcing function, and no artifact at the end.

This program supplies the three things a university actually provides that a playlist does not: **an order**, **a standard**, and **an artifact**. The lectures you can get free. The structure is what has been missing.

Design principles

- **Build, don't watch.** Every module ends in code that runs. A module you cannot demonstrate is a module you have not finished.
- **One renderer, four semesters.** The track courses compound into a single codebase rather than twelve disconnected toy projects.
- **Theory where it pays rent.** Core courses are included because graphics work breaks on them — cache behaviour, scheduling, asymptotics — not for completeness.
- **Free, permanently.** Every external resource is open courseware, an author-released book, a public lecture recording, or a free-audit MOOC. No resource in this program requires payment.

2 Admission and prerequisites

Two mathematics prerequisites and one programming prerequisite are assumed. They are not part of the 36 credits. If you are confident in all three, begin at Semester 1; if not, spend four to six weeks here first — skipping them is the single most common cause of failure in Semester 1.

Prerequisite	Why it is required	Free resource
Discrete mathematics & proof	CSCE 629 is unreadable without proof technique, induction, and asymptotic reasoning.	MIT 6.042J / 6.1200J Mathematics for CS (OCW, full video)
Linear algebra	The graphics pipeline is linear algebra. Bases, change of basis, projection, and eigenstructure are used daily from week 2.	MIT 18.06 — Strang (OCW) · 3Blue1Brown, Essence of Linear Algebra
C++ fluency	Every track project is C++ with a graphics API. You need RAII, templates, and comfort with manual memory.	learncpp.com (complete, free)

3 Degree structure

Twelve courses, thirty-six credit-equivalents, plus a capstone. Five core courses establish the systems and theory foundation; six track courses build graphics depth; one computation elective covers the parallel hardware all modern rendering runs on.

Core — 5 courses, 15 credits

Course	Title	Why it is in a graphics degree
CSCE 629	Analysis of Algorithms	Spatial data structures, BVH construction, and visibility are algorithm-design problems with hard asymptotic limits.
CSCE 614	Computer Architecture	Cache lines, memory bandwidth, and SIMD decide real frame times far more often than algorithmic cleverness does.
CSCE 611	Operating Systems	Scheduling, virtual memory, and synchronisation underlie every render thread, streaming system, and frame pacing bug.
CSCE 606	Software Engineering	An engine is a long-lived codebase with many contributors. This is the course that keeps it from collapsing.
CSCE 608	Database Systems	Asset pipelines, scene databases, and streaming are indexing and query-planning problems wearing different clothes.

Graphics & Game Engines track — 6 courses, 18 credits

Course	Title	What you build
CSCE 641	Computer Graphics	A software rasteriser, then a real-time GPU pipeline: transforms, shading, texturing, shadows.
CSCE 645	Geometric Modeling	A mesh-processing library: splines, subdivision, simplification, parameterisation, remeshing.
CSCE 647	Image Synthesis	A physically based path tracer with importance sampling, multiple importance sampling, and participating media.
CSCE 649	Physically-Based Modeling	A simulation layer: particles, rigid bodies, cloth, position-based dynamics, and a FLIP fluid.
CSCE 650	Virtual Reality	A stereo renderer with tracking, lens correction, and a latency budget you measure rather than guess.
CSCE 748	Computational Photography	An imaging pipeline: RAW processing, HDR merge, tonemapping, and lightfield reconstruction.

Computation elective — 1 course, 3 credits

Course	Title	What you build
CSCE 735	Parallel Computing	GPU compute kernels: a parallel BVH builder and a compute-shader particle system, profiled and optimised.

Capstone — UC 691

A sustained research-and-build project spanning Semesters 3 and 4, carried out in a real engine codebase. The deliverable is a working system, a written technical report in the format of a graphics conference paper, and a recorded demonstration. This is the artifact that stands in for the credential.

4 Four-semester sequence

Each semester is fifteen weeks: thirteen teaching modules plus two weeks for projects and review. Three courses at a time is a full load at roughly 25–35 hours per week. At half pace — one or two courses per term — the program takes three years instead of two, and that is a perfectly good way to do it.

Semester	Courses
Semester 1 — Foundations	CSCE 629 Analysis of Algorithms CSCE 614 Computer Architecture CSCE 641 Computer Graphics
Semester 2 — Systems & geometry	CSCE 611 Operating Systems CSCE 606 Software Engineering CSCE 645 Geometric Modeling
Semester 3 — Light & motion	CSCE 608 Database Systems CSCE 647 Image Synthesis CSCE 649 Physically-Based Modeling <i>Capstone begins: literature review and proposal.</i>
Semester 4 — Perception & parallelism	CSCE 650 Virtual Reality CSCE 748 Computational Photography CSCE 735 Parallel Computing <i>Capstone: implementation, report, demonstration.</i>

THE THROUGH-LINE

Semester 1 gets triangles on screen and teaches you to reason about cost. Semester 2 gives you the geometry to feed the renderer and the engineering discipline to keep it alive. Semester 3 replaces approximations with physics — real light transport, real dynamics. Semester 4 closes the loop to human perception and to the parallel hardware that makes any of it real time.

By the end you do not have twelve projects. You have one engine, and you understand every layer of it.

5 How each course is delivered

Every course in this program ships as a folder containing four kinds of document. The slide decks and notes are original material written for this program; the resource map points at the free university courseware that covers the same ground in a different voice.

Document	What it is	How to use it
Syllabus (PDF)	Learning outcomes, the thirteen-module schedule, projects, and the assessment standard.	Read once at the start of the course; revisit at each project deadline.

Document	What it is	How to use it
Resource map (PDF)	Module-by-module mapping onto free lectures, books, and problem sets, with specific lecture numbers where possible.	This is your lecture hall. Watch the mapped lecture, then read the module notes.
Module slides (PPTX)	One deck per module — the lecture, in the form you would see it in a classroom. Editable.	Work through before or after the external lecture. Speaker notes carry the argument the slide compresses.
Module notes (PDF)	A dense written treatment with derivations, worked examples, code, exercises, and a self-check.	The reference you return to. Do the exercises — they are the assessment.

Assessment without a registrar

Nobody is going to grade you, so the standard has to be mechanical and honest. A module is complete when all three hold:

- 1 **The code runs** and produces the output the module specifies — an image, a measurement, a passing test.
- 2 **The self-check is answered from memory**, in writing, without looking at the notes. Writing it down is not optional; recognition feels like knowledge and is not.
- 3 **You can explain the failure modes** — what breaks the method, and what you would reach for instead.

A course is complete when its projects are in the portfolio with written notes on what was hard. Keep the portfolio in public version control from day one. In this field the repository *is* the transcript, and it is a better one.

6 Sources and licensing

Two categories of material appear in this program, under different terms. The distinction matters, so it is stated plainly.

Material	Origin	Terms
Slides, notes, syllabi, this handbook	Written originally for the University of Claude.	Free to copy, adapt, redistribute, and sell. No attribution required. No rights reserved.
Linked courses, books, lectures, labs	MIT OCW, Stanford, CMU, Berkeley, ETH, edX, and individual authors who published them free.	Each remains under its own licence. Linked, never reproduced. Read each site's terms before redistributing anything from it.

Student-supplied sources

The following were nominated for inclusion and are mapped into the curriculum at the courses indicated:

- UC San Diego CSE167x Computer Graphics (edX) — *CSCE 641*
- MIT 6.5830 Database Systems, Fall 2023 — *CSCE 608*
- MIT 6.S081 Operating System Engineering, 2020 — *CSCE 611*
- MIT 6.824 / 6.5840 Distributed Systems — *CSCE 662, 678*
- MIT 6.034 Artificial Intelligence, Fall 2010 — *CSCE 625*
- MIT 6.S191 Introduction to Deep Learning — *CSCE 636*
- MIT 6.006 Introduction to Algorithms, Spring 2020 — *CSCE 629*
- MIT 6.004 / MITx 6.004.1x Computation Structures — *CSCE 614*
- MIT 6.1200J Mathematics for Computer Science — *prerequisite*
- Stanford CS145 Introduction to Databases (archived) — *CSCE 603*
- Stanford CS144 Introduction to Computer Networking — *CSCE 612*
- Stanford CS229 Machine Learning — *CSCE 633*
- Stanford CS231n CNNs for Visual Recognition — *CSCE 636*
- UC Berkeley CS287 Advanced Robotics — *CSCE 635, 752*
- ETH Zürich — Onur Mutlu, Computer Architecture — *CSCE 614*
- RIT Cybersecurity Fundamentals (edX) — *CSCE 701*
- Harvard CS50 and CS50 AI with Python — *CSCE 706, 708, 625*
- Neso Academy, Compiler Design — *CSCE 605*
- OSSU Open Source Computer Science — *general reference*

Two further nominated sources are curricula rather than courses — [OSSU Open Source Computer Science](#) and [mrcomputerscience.com](#) — and are worth reading alongside this handbook as alternative degree structures. The breadth-requirement model used here follows the common American graduate pattern of

core breadth plus a declared specialisation.

7 Where to begin

- 1 Verify the three prerequisites honestly. Fix gaps now, not in week 6.
- 2 Open *Courses/CSCE641-Computer-Graphics/* and read the syllabus.
- 3 Create the portfolio repository. First commit before first lecture.
- 4 Start Module 1. Build something that draws a triangle by week 2.